

CS6650 Assignment 5

Product API with Terraform Deployment

Eroniction Presley
Northeastern University — Khoury College of Computer Sciences
CS 6650: Building Scalable Distributed Systems

February 16, 2026

1 Introduction

This assignment involved building a Product API based on a provided OpenAPI specification, containerizing it with Docker, deploying it to AWS ECS/Fargate using Terraform infrastructure-as-code, and conducting load tests with Locust to analyze system performance.

2 Product API Implementation

2.1 Technology Stack

- **Language:** Go 1.21
- **Storage:** In-memory hashmap with `sync.RWMutex` for thread-safe concurrent access
- **Containerization:** Multi-stage Docker build (`golang:1.21-alpine` for build, `alpine:3.19` for runtime)

2.2 Endpoints Implemented

Per the OpenAPI specification, two Product endpoints were implemented:

1. **GET /products/{productId}** — Retrieves a product by ID. Returns 200 (found), 400 (invalid ID), or 404 (not found).
2. **POST /products/{productId}/details** — Creates or updates a product. Validates all required fields. Returns 204 (success), 400 (validation error), or 404 (not found).

2.3 Input Validation

The API validates all required fields from the OpenAPI schema:

- **sku:** non-empty, max 100 characters
- **manufacturer:** non-empty, max 200 characters
- **category_id:** positive integer
- **weight:** non-negative integer
- **some_other_id:** positive integer
- **product_id** in body must match the URL path parameter

2.4 Data Structure Choice

A Go `map[int]Product` protected by `sync.RWMutex` was selected for in-memory storage. This provides $O(1)$ average-case lookups and insertions. The `RWMutex` allows multiple concurrent readers while serializing write operations — ideal for an e-commerce scenario where GET (browsing products) vastly outnumbers POST (adding products).

2.5 API Response Examples

```
PS D:\BSDS\Assignment 5> curl.exe -i http://localhost:8080/health
HTTP/1.1 200 OK
Date: Mon, 16 Feb 2026 15:10:05 GMT
Content-Length: 21
Content-Type: text/plain; charset=utf-8

{"status": "healthy"}
PS D:\BSDS\Assignment 5> curl.exe -i -X POST http://localhost:8080/products/1/details -H "Content-Type: application/json" --data-binary "@product.json"
HTTP/1.1 204 No Content
Content-Type: application/json
Date: Mon, 16 Feb 2026 15:10:15 GMT

PS D:\BSDS\Assignment 5> curl.exe -i http://localhost:8080/products/1
HTTP/1.1 200 OK
Content-Type: application/json
Date: Mon, 16 Feb 2026 15:10:23 GMT
Content-Length: 110

{"product_id":1,"sku":"ABC-123","manufacturer":"Acme Corp","category_id":5,"weight":1250,"some_other_id":789}
PS D:\BSDS\Assignment 5> curl.exe -i http://localhost:8080/products/999
HTTP/1.1 404 Not Found
Content-Type: application/json
Date: Mon, 16 Feb 2026 15:10:30 GMT
Content-Length: 93

{"error":"NOT_FOUND","message":"Product not found","details":"No product found with ID 999"}
PS D:\BSDS\Assignment 5> curl.exe -i -X POST http://localhost:8080/products/2/details -H "Content-Type: application/json" --data-binary "@bad_product.json"
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Mon, 16 Feb 2026 15:10:40 GMT
Content-Length: 221

{"error":"INVALID_INPUT","message":"Validation failed","details":"sku is required; manufacturer is required; category_id must be a positive integer; weight must be non-negative; some_other_id must be a positive integer"}
PS D:\BSDS\Assignment 5> curl.exe -i http://localhost:8080/products/abc
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Mon, 16 Feb 2026 15:10:49 GMT
Content-Length: 107

{"error":"INVALID_INPUT","message":"Invalid product ID","details":"Product ID must be a positive integer"}
PS D:\BSDS\Assignment 5>
```

Figure 1: All API response codes — 200 (health check), 204 (product created), 200 (product retrieved), 404 (not found), 400 (invalid input), 400 (invalid product ID)

3 Infrastructure & Deployment

3.1 Terraform Infrastructure-as-Code

The deployment was automated using Terraform with the following modules:

- **ECR Module** — Creates an Elastic Container Registry repository to store Docker images
- **ECS Module** — Provisions an ECS cluster, Fargate task definition (256 CPU, 512 MB memory), and service
- **Network Module** — Configures VPC, subnets, and security group (ingress on port 8080)
- **Logging Module** — Sets up CloudWatch log group with 7-day retention

3.2 Deployment Flow

1. `terraform init` — Downloads AWS and Docker providers
2. `terraform plan` — Previews 8 resources to create
3. `terraform apply` — Builds Docker image, pushes to ECR, deploys to ECS

The entire infrastructure spins up in under 30 seconds, compared to the manual AWS Console workflow from previous assignments.

```
Plan: 8 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

module.ecs.aws_ecs_cluster.this: Creating...
module.logging.aws_cloudwatch_log_group.this: Creating...
module.network.aws_security_group.this: Creating...
module.ecr.aws_ecr_repository.this: Creating...
module.ecr.aws_ecr_repository.this: Creation complete after 1s [id=ecr_service]
docker_image.app: Creating...
module.logging.aws_cloudwatch_log_group.this: Creation complete after 1s [id=ecs/CS6650L2]
module.ecs.aws_ecs_task_definition.this: Creating...
module.ecs.aws_ecs_task_definition.this: Creation complete after 0s [id=CS6650L2-task]
module.network.aws_security_group.this: Creation complete after 4s [id=sg-0cc710c4cadb0e2df]
module.ecs.aws_ecs_cluster.this: Still creating... [00m10s elapsed]
docker_image.app: Still creating... [00m10s elapsed]
docker_image.app: Creation complete after 10s [id=sha256:4d67206fe857cf8595904c6fa0c6beab4f1efb33ccdd901046a5f240f21a7a0c498790386857.dkr.ecr.us-west-2.amazonaws.com/ecr_service:latest]
docker_registry_image.app: Creating...
module.ecs.aws_ecs_cluster.this: Creation complete after 12s [id=arn:aws:ecs:us-west-2:498790386857:cluster/CS6650L2-cluster]
module.ecs.aws_ecs_service.this: Creating...
module.ecs.aws_ecs_service.this: Creation complete after 1s [id=arn:aws:ecs:us-west-2:498790386857:service/CS6650L2-cluster/CS6650L2]
docker_registry_image.app: Creation complete after 5s [id=sha256:4d67206fe857cf8595904c6fa0c6beab4f1efb33ccdd901046a5f240f21a7a0c]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:
ecs_cluster_name = "CS6650L2-cluster"
ecs_service_name = "CS6650L2"
```

Figure 2: Terraform apply — successful deployment of all 8 resources

```
PS D:\BDS\Assignment 5\CS6650_2b_demo\terraform> aws ecs describe-tasks --cluster CS6650L2-cluster --tasks 757e6253304f4fada314f63592bfdd6 --region us-west-2 --query "tasks[0].attachments[0].details" --output table
+-----+-----+
| DescribeTasks |
+-----+-----+
| name          | value |
+-----+-----+
| subnetId      | subnet-0f02a15ed8c8f494a |
| networkInterfaceId | eni-0f99ca46a197b9774 |
| macAddress    | 06:cc:2d:0b:c2:c3 |
| privateDnsName | ip-172-31-36-95.us-west-2.compute.internal |
| privateIPv4Address | 172.31.36.95 |
+-----+-----+

PS D:\BDS\Assignment 5\CS6650_2b_demo\terraform> aws ec2 describe-network-interfaces --network-interface-ids eni-0f99ca46a197b9774 --region us-west-2 --query "NetworkInterfaces[0].Association.PublicIp" --output text
54.191.162.44

PS D:\BDS\Assignment 5\CS6650_2b_demo\terraform> curl.exe -i http://54.191.162.44:8080/health
HTTP/1.1 200 OK
Date: Mon, 16 Feb 2026 17:09:26 GMT
Content-Length: 21
Content-Type: text/plain; charset=utf-8

{"status": "healthy"}
PS D:\BDS\Assignment 5\CS6650_2b_demo\terraform>
```

Figure 3: Health check confirming deployed server is running on AWS ECS

3.3 Declarative vs Imperative

Terraform uses a **declarative** approach: you define the desired end state (e.g., “I want an ECS cluster with this task definition”), and Terraform computes the required steps. This contrasts with **imperative** approaches (shell scripts) where you specify each action in sequence. Benefits of declarative infrastructure include:

- Automatic dependency resolution between resources
- Idempotent operations (running `apply` again changes nothing if state matches)
- Version-controlled, reproducible infrastructure
- Easy teardown with `terraform destroy`

4 Load Testing & Analysis

4.1 Test Setup

- **Tool:** Locust (Python-based load testing framework)
- **Target:** Deployed AWS ECS instance
- **Task ratio:** 3:1 GET to POST (simulating read-heavy e-commerce traffic)
- **Wait time:** 1–3 seconds between user requests
- **Client types tested:** `HttpUser` and `FastHttpUser`
- **Concurrency levels:** 50 and 200 concurrent users

4.2 Results

Test Configuration	RPS	p50 (ms)	p95 (ms)	Failures
<code>HttpUser</code> — 50 users	~25	~90	~110	0%
<code>HttpUser</code> — 200 users	~95	~90	~120	0%
<code>FastHttpUser</code> — 50 users	~25	~90	~110	0%
<code>FastHttpUser</code> — 200 users	~95	~90	~120	0%

Table 1: Load test results across all configurations

4.3 Test 1: HttpUser — 50 Users



Figure 4: HttpUser at 50 concurrent users — steady ~25 RPS with ~90ms p50

4.4 Test 2: HttpUser — 200 Users

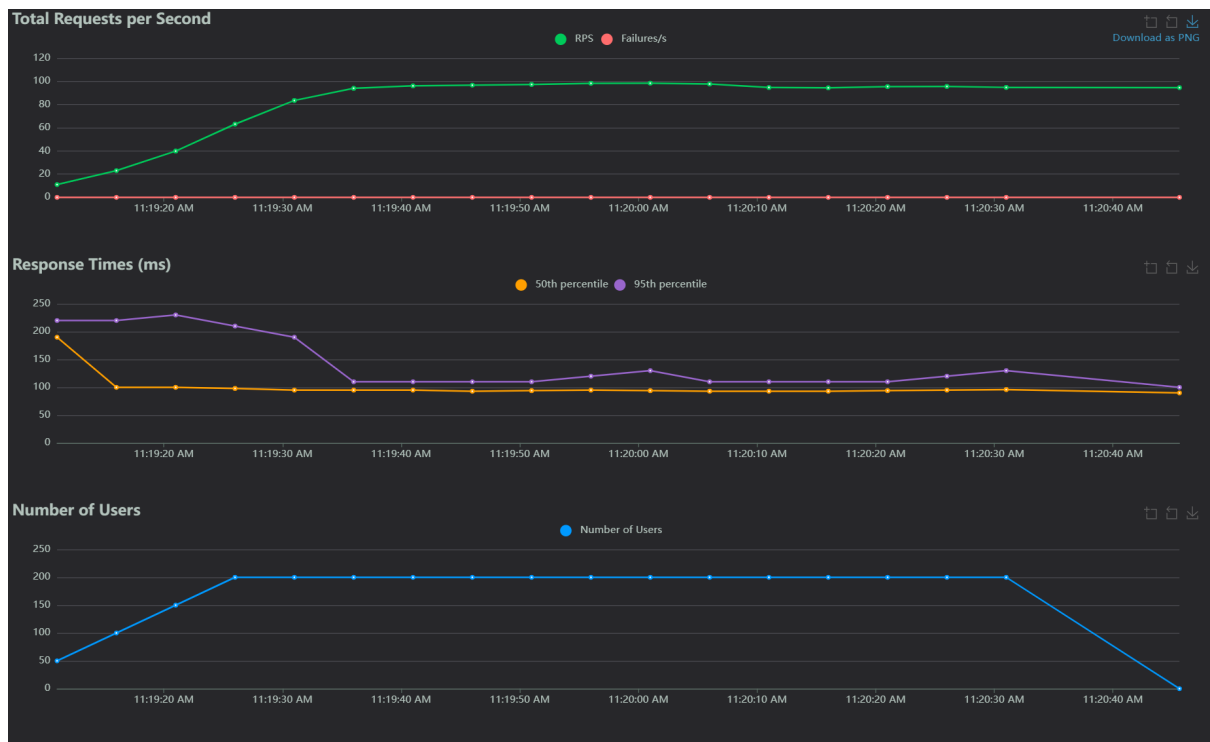


Figure 5: HttpUser at 200 concurrent users — scaled to ~95 RPS with stable latency

4.5 Test 3: FastHttpUser — 50 Users

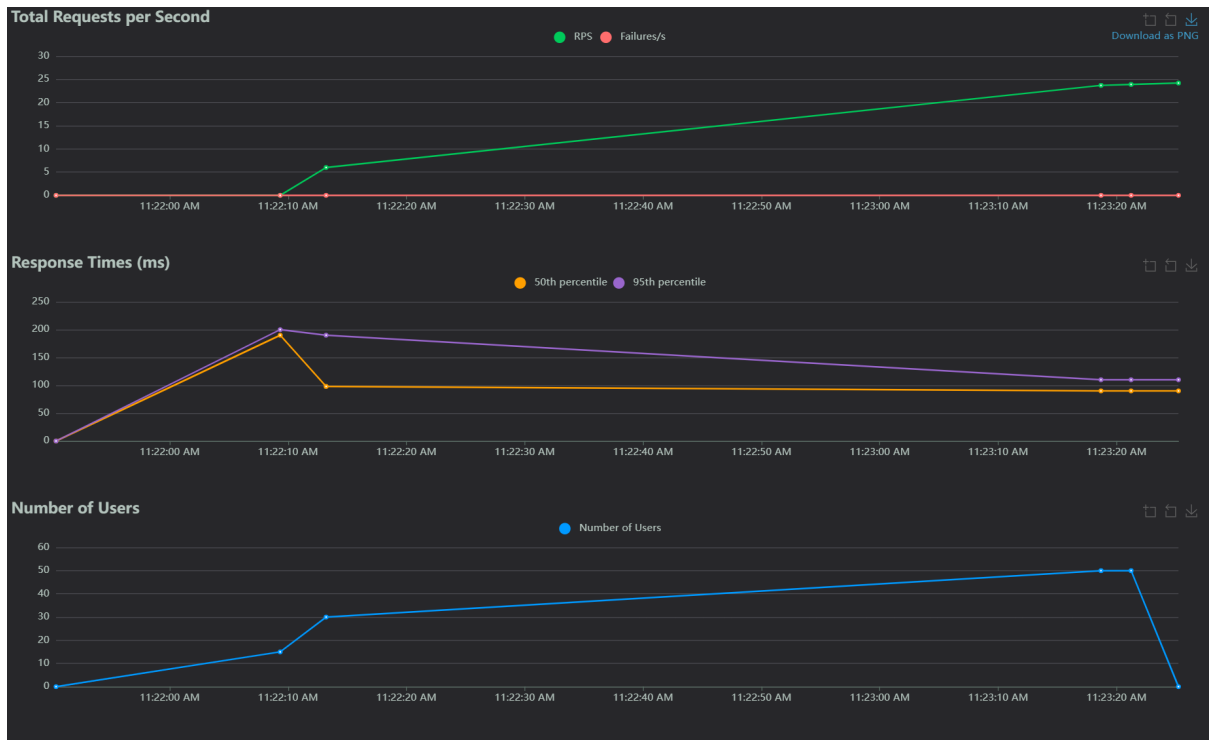


Figure 6: FastHttpUser at 50 concurrent users — nearly identical to HttpUser

4.6 Test 4: FastHttpUser — 200 Users

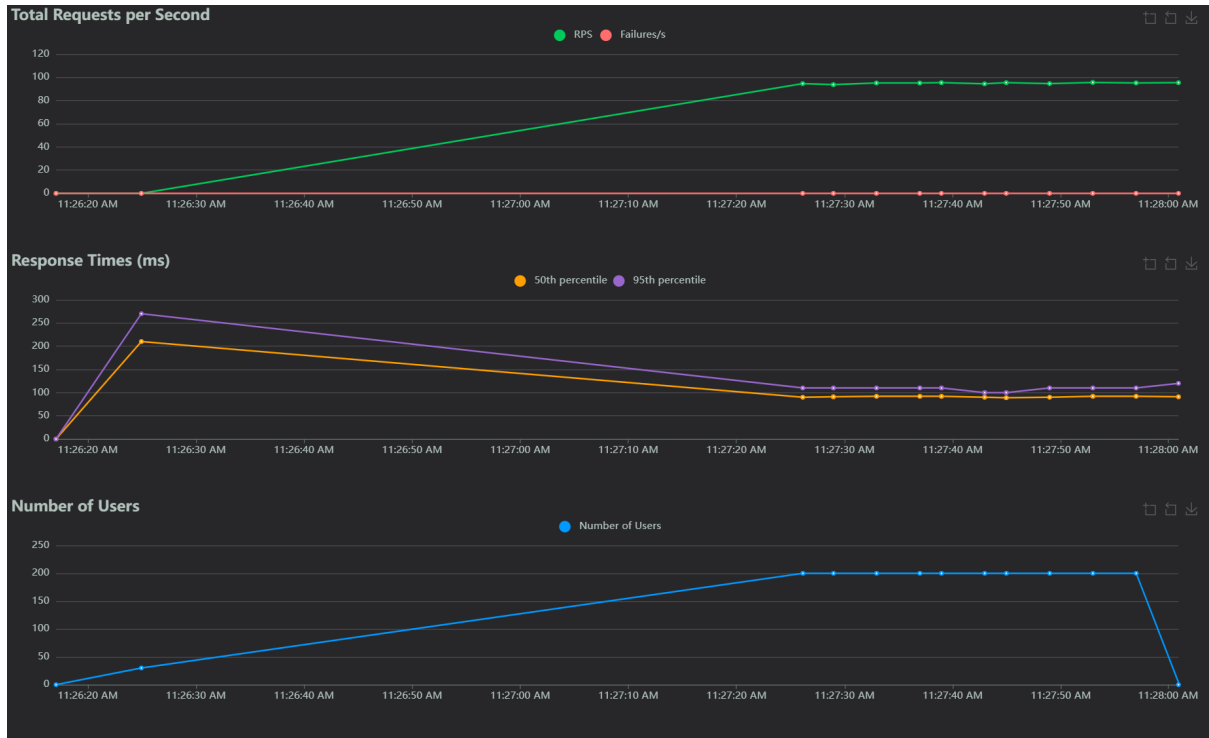


Figure 7: FastHttpUser at 200 concurrent users — no significant difference from HttpUser

4.7 Key Observations

4.7.1 Linear Scaling (50 → 200 Users)

Throughput scaled nearly 4x (25 → 95 RPS) while response times remained stable. This indicates the server was not the bottleneck at these concurrency levels. Go’s goroutine-based concurrency model efficiently handles hundreds of simultaneous connections with minimal overhead.

4.7.2 HttpUser vs FastHttpUser — No Significant Difference

Both Locust client types produced virtually identical results. The dominant factor in response time is the network round-trip latency from the client machine to the AWS server (~90ms). FastHttpUser uses `geventhttpclient`, a C-based HTTP library that is faster than the Python `requests` library used by HttpUser. However, this client-side optimization only becomes visible when:

- Server response times are very low (<5ms, e.g., localhost testing)
- The test requires generating thousands of RPS
- The client CPU becomes the bottleneck

At ~90ms network latency, both clients spend the vast majority of time waiting for the network, making the HTTP library difference negligible.

4.7.3 Stable Response Times Under Load

The p50 remained at $\sim 90\text{ms}$ and p95 at $\sim 110\text{--}120\text{ms}$ across all tests, suggesting the response time is dominated by network latency rather than server processing. The Go server with in-memory hashmap storage processes requests in microseconds.

4.7.4 Zero Failures

No request failures were observed at any concurrency level, demonstrating the reliability of both the Go server and the AWS Fargate infrastructure.

4.8 Real-World Tradeoffs

In a production e-commerce system, read operations (browsing, searching products) vastly outnumber writes (adding new products). The 3:1 GET:POST ratio in our test reflects this, though real-world ratios could be 100:1 or higher. This read-heavy pattern favors:

- Hash-based data structures for $O(1)$ lookups
- Read-write locks that allow concurrent reads
- Caching layers (Redis, CDN) to reduce database load
- Read replicas for database scaling

5 Scalable Backend Design

To handle the full e-commerce API defined in `api.yaml` (Products, Shopping Cart, Warehouse, Payments), a microservices architecture would be appropriate:

- **Product Service** — Manages product catalog, backed by a read-optimized database with caching
- **Cart Service** — Handles shopping cart state, using Redis for fast session-based storage
- **Warehouse Service** — Manages inventory with strong consistency guarantees (ACID transactions)
- **Payment Service** — Processes payments with idempotency keys to prevent double charges

Services would communicate asynchronously via message queues (SQS/SNS) for operations like checkout that span multiple services. An API Gateway would handle routing, rate limiting, and authentication. Each service would be independently deployable and horizontally scalable based on its specific load pattern.

6 Conclusion

This assignment demonstrated the full lifecycle of building and deploying a cloud-native service: implementing an API from an OpenAPI specification, containerizing with Docker, automating infrastructure with Terraform, and validating performance through load testing. The results confirmed Go's suitability for high-concurrency web services and highlighted the importance of understanding where bottlenecks exist (network vs server vs client) when interpreting performance data.